ARHITECTURA SISTEMELOR DE CALCUL

Cheat sheet Assembly x86 (AT&T, 32 biți)

4 noiembrie 2026 · Alessandra

Parte	Conține
I. Bazele	1 Scheletul programului · 2 Sintaxa AT&T · 3 Registre · 4 Date în `.data`
II. Instrucțiuni	5 Instrucțiuni de bază (`mul`, `div`) · 6 Comparații și salturi (`cmpl`, `testl`)
III. Vectori	7 Adresarea elementelor · 8 Parcurgerea unui vector
IV. Șabloane	9 Maxim, sume, elemente impare
V. Greșeli	10 Greșeli frecvente · 11 Recapitulare: ce trebuie să încapă pe foaie

PARTEA I BAZELE

1. Scheletul obligatoriu al programului

Orice program are zona de date, zona de cod, punctul de intrare `main` și ieșirea prin apel de sistem. Fără ieșire, programul continuă în memorie după ultima instrucțiune și dă segmentation fault.

```
.data                   # variabile și vectori
    n: .long 5
    v: .long 3, 8, 5, 10, 7

.text                   # codul
.global main
main:
    # ... instrucțiunile tale ...

et_exit:
    movl $1, %eax       # apel de sistem EXIT
    xorl %ebx, %ebx     # cod de ieșire 0
    int $0x80
```

2. Sintaxa AT&T

Regulile de sintaxă AT&T, de ținut minte:

- Ordinea este **sursă, destinație**: `movl x, %eax` copiază x în eax.
- Registrele au prefix `%` (`%eax`), constantele au prefix `$` (`$10`).
- Sufixul indică dimensiunea: `b` = 1 octet, `w` = 2 octeți, `l` = 4 octeți (`movb`, `movw`, `movl`).
- Comentariile încep cu `#`. Etichetele se termină cu `:`.
- `x` înseamnă **valoarea** variabilei din memorie. `$x` înseamnă **adresa** ei.

3. Registre

Pe 32 de biți avem registrele generale `eax`, `ebx`, `ecx`, `edx`, `esi`, `edi` (pe 32 de biți). Primele patru se pot accesa și parțial.

32 biți	16 biți	8 biți sus	8 biți jos
`%eax`	`%ax`	`%ah`	`%al`
`%ebx`	`%bx`	`%bh`	`%bl`
`%ecx`	`%cx`	`%ch`	`%cl`
`%edx`	`%dx`	`%dh`	`%dl`

`%esi` și `%edi` au doar partea de 16 biți (`%si`, `%di`).

Roluri uzuale

Roluri uzuale, care ajută la citirea codului:

- `%eax`: rezultate, operand implicit la `mul` și `div`.
- `%edx`: partea de sus a rezultatului la `mul`, restul la `div`.
- `%ecx`: contorul pentru `loop`.
- `%edi` / `%esi`: indecși pentru parcurgerea vectorilor.

> **Registrele parțiale**
>
> Scrierea într-o parte a registrului **nu atinge restul**: `movb $5, %al` schimbă doar cel mai mic octet din `%eax`. Deci dacă vreți să schimbați conținutul unui registru, **NU** folosiți registrele parțiale!

> **Despre %ebp și %esp**
>
> Dacă vă veți uita pe suportul de laborator sau dacă faceți extensive research pentru registre, veți vedea că există (și veți folosi mai departe în curs) registrele `%ebp` și `%esp`. Pe aceștia **NU** îi veți folosi când vine vorba de acest gen de cod (parcurgere de vector, stocare de variabile etc). Ei sunt folosiți pentru lucru cu **STIVA** (veți apela cu ajutorul lor funcții, veți stoca argumente de funcții și așa mai departe).
>
> De multe ori, dacă nu îi veți folosi în acest context, veți primi direct segfault din cauza faptului că stricați structura stivei (nu se întâmplă mereu, dar atunci când nu se întâmplă puteți considera doar că ați avut mare noroc).

4. Date în `.data` și dimensiuni

Directivă	Dimensiune	Exemplu
`.byte`	1 octet (8 biți)	`b: .byte 'a', 7`
`.word`	2 octeți (16 biți)	`w: .word 13, 260`
`.long`	4 octeți (32 biți)	`x: .long 0x8fde2025`
`.ascii`	un octet pe caracter	`s: .ascii "Sir\n"` (4 octeți)
`.asciz`	caractere + 1 (terminatorul `\0`)	`s: .asciz "Sir\n"` (5 octeți)
`.space N`	N octeți, inițializați cu 0	`buf: .space 40`

Vectorii în memorie

- Elementele sunt **consecutive** în memorie, de la adresa etichetei.
- Un vector de `.long` cu n elemente ocupă 4·n octeți. Elementul i este la adresa `v + 4·i`.
- Numărul de elemente se poate păstra într-o variabilă separată: `n: .long 5`.

> **Valoare sau adresă?**
> Se citesc cu `movl x, %eax` (valoarea) sau `movl $x, %eax` (adresa).

PARTEA II INSTRUCȚIUNI

5. Instrucțiuni de bază

Instrucțiune	Efect	Exemplu
`movl s, d`	d = s (nu se poate memorie → memorie)	`movl x, %eax`
`addl s, d`	d = d + s	`addl $4, %ebx`
`subl s, d`	d = d − s	`subl %eax, %ebx`
`incl d` / `decl d`	d++ / d−−	`incl %edi`
`xorl %r, %r`	pune valoarea 0 în registru	`xorl %edx, %edx`
`mull s`	EDX:EAX = EAX · s (fără semn)	`mull %ebx`
`divl s`	EAX = EDX:EAX / s (cât), EDX = rest	`divl %ecx`
`lea a, %r`	încarcă adresa a în registru	`lea v, %esi`

La `mul` și `div`

La `mul` și `div`:

- Operandul implicit este `%eax`. Operandul scris pe instrucțiune nu poate fi constantă (`mull $5` nu merge, se pune 5 întâi într-un registru).
- **Înainte de `divl` se pune `%edx` pe 0**: `xorl %edx, %edx`. Altfel împărțirea folosește un EDX vechi și poate da eroare (SIGFPE).
- Pentru paritate sau mod 2: `movl $2, %ecx`, `divl %ecx`, iar restul din `%edx` este 0 (par) sau 1 (impar).

6. Comparații și salturi

`cmpl a, b` calculează b − a, **nu** modifică operanzii și setează doar flagurile. Instrucțiunea de salt de după ea citește flagurile. Atenție la ordine în AT&T: `cmpl %ebx, %eax` urmat de `jg` sare dacă **%eax > %ebx**.

`testl a, b` face `b AND a`, nu salvează rezultatul, dar setează flagurile (ZF = 1 dacă rezultatul este 0).

Salt	Sare dacă (după cmpl a, b)	Interpretare
`jmp et`	întotdeauna	salt necondiționat
`je` / `jz`	b = a	egal / zero
`jne` / `jnz`	b ≠ a	diferit / nenul
`jg`	b > a	cu semn
`jge`	b ≥ a	cu semn
`jl`	b < a	cu semn
`jle`	b ≤ a	cu semn
`ja` / `jb`	b > a / b < a	fără semn

Tipare de ținut minte

```
testl $1, %eax      # verifică bitul 0
jz et_par           # sare dacă bitul 0 = 0, deci număr par
                    # (continuă aici dacă e impar)

testl %eax, %eax    # eax = 0 ?
jz et_zero

cmpl $0, %ecx       # compară cu constanta
je et_gata
```

Structura if

O structură `if` se scrie cu salt peste bloc, pe condiția **inversă**: dacă vrei să execuți un bloc doar când eax > ebx, sari peste el cu `jle`.

PARTEA III VECTORI

7. Adresarea elementelor unui vector

Adresarea cu index are forma `v(,%edi,4)` și înseamnă **adresa v + %edi · 4**. Numărul 4 este dimensiunea unui element (`.long` = 4, `.word` = 2, `.byte` = 1). Indexul începe de la 0.

Vrei	Scrii
elementul cu indexul 0	`movl v, %eax`
elementul cu indexul i (i în `%edi`)	`movl v(,%edi,4), %eax`
elementul cu indexul 3, fix	`movl v+12, %eax`
ultimul element (n în `%ecx`)	`movl v-4(,%ecx,4), %eax`

Alte moduri de adresare

Există de asemenea mai multe moduri de adresare a unui vector. Cea mai cunoscută (cea prezentată mai sus) este, din punctul meu de vedere, și cea mai ușoară (+ vă salvează un registru pe care îl puteți folosi mai departe în alte calcule). Aici vă atașez și celelalte moduri de adresare:

Scriere	Adresa calculată	Când o folosești
`v(,%edi,4)`	v + edi·4	vector cunoscut după nume, cu index
`(%esi,%edi,4)`	esi + edi·4	adresa vectorului este într-un registru
`v(%esi,%edi,4)`	v + esi + edi·4	combinat, mai rar
`(%esi)`	esi	un singur element, la adresa din registru
`4(%esi)`	esi + 4	elementul următor (deplasament fix)
`(%esi,%edi)`	esi + edi	scală omisă = 1, util la `.byte`

Încărcarea adresei în registru se face în două moduri: `movl $v, %esi` sau `lea v, %esi`. Atenție: `movl v, %esi` ar încărca **valoarea** primului element, nu adresa.

Reguli de ținut minte

Baza și indexul sunt registre de **32 de biți** (`%eax`...`%edi`), scala poate fi doar 1, 2, 4 sau 8, iar `%esp` nu poate fi index.

8. Parcurgerea unui vector

Există trei moduri de parcurgere, la alegere: A (index și `loop`), B (index comparat cu n) și C (pointer).

Varianta A: cu index și `loop`

Contorul `%ecx` pornește de la n, iar `loop` scade `%ecx` cu 1 și sare dacă nu a ajuns la 0.

```
    movl n, %ecx            # contor = n
    movl $0, %edi           # index = 0
et_loop:
    movl v(,%edi,4), %eax   # eax = v[edi]
    # ... prelucrarea lui eax ...
    incl %edi
    loop et_loop            # ecx--, repetă cât timp ecx != 0
```

Varianta B: cu index comparat de n, fără `loop`

```
    movl $0, %edi
et_loop:
    cmpl n, %edi            # edi - n
    jge et_exit             # dacă edi >= n, am terminat
    movl v(,%edi,4), %eax
    # ... prelucrarea lui eax ...
    incl %edi
    jmp et_loop
```

Varianta C: cu pointer

Parcurgere cu pointer (fără index, avansezi direct adresa):

```
    movl $v, %esi
    movl n, %ecx
et_loop:
    movl (%esi), %eax       # eax = elementul curent
    # ... prelucrarea ...
    addl $4, %esi           # următorul element (.long = 4 octeți)
    loop et_loop
```

Puncte care se pierd des

- `loop` cu `%ecx` = 0 la intrare rulează de aproximativ 4 miliarde de ori. Se folosește doar dacă n ≥ 1.
- `loop` modifică `%ecx`, deci nu se folosește `%ecx` și pentru altceva în buclă (de exemplu pentru `div`).
- `incl %edi` trebuie pus pe toate căile de execuție ale buclei, inclusiv pe cea care sare peste prelucrare.

9. Șabloane complete

Toate presupun în `.data` un vector `v` de `.long` și lungimea lui `n` (`.long`). Rezultatul rămâne în `%ebx`. Cele patru tipare de mai jos acoperă aproape orice cerință scurtă: un acumulator, o parcurgere și o condiție.

Maximul dintr-un vector

Se pornește cu primul element ca maxim, apoi se parcurge de la indexul 1.

```
main:
    movl v, %ebx                # max = v[0]
    movl $1, %edi               # index = 1
et_loop:
    cmpl n, %edi
    jge et_exit
    movl v(,%edi,4), %eax       # eax = v[i]
    cmpl %ebx, %eax             # eax - ebx
    jle et_next                 # eax <= max: nu schimbăm
    movl %eax, %ebx             # max = eax
et_next:
    incl %edi
    jmp et_loop
```

Suma elementelor de pe poziții impare

Aici contează convenția: dacă pozițiile se numără de la 1 (primul, al treilea, ...) indexurile sunt 0, 2, 4, ...; dacă se numără de la 0, indexurile sunt 1, 3, 5, ... Se pornește de la indexul corespunzător și se avansează cu 2.

```
main:
    xorl %ebx, %ebx             # suma = 0
    movl $0, %edi               # index de start (0 sau 1)
et_loop:
    cmpl n, %edi
    jge et_exit
    addl v(,%edi,4), %ebx       # suma += v[i]
    addl $2, %edi               # sărim din 2 în 2
    jmp et_loop
```

Suma elementelor care au valoare impară

```
main:
    xorl %ebx, %ebx             # suma = 0
    xorl %edi, %edi             # index = 0
et_loop:
    cmpl n, %edi
    jge et_exit
    movl v(,%edi,4), %eax
    testl $1, %eax              # bitul 0
    jz et_next                  # par: sărim
    addl %eax, %ebx             # impar: adunăm
et_next:
    incl %edi
    jmp et_loop
```

Maximul elementelor impare (elementele sunt pozitive)

Maximul pornește de la 0, ceea ce e corect pentru că toate elementele sunt pozitive și există cel puțin un element impar.

```
main:
    xorl %ebx, %ebx             # max = 0
    xorl %edi, %edi
et_loop:
    cmpl n, %edi
    jge et_exit
    movl v(,%edi,4), %eax
    testl $1, %eax
    jz et_next                  # par: sărim
    cmpl %ebx, %eax
    jle et_next                 # nu e mai mare decât max
    movl %eax, %ebx
et_next:
    incl %edi
    jmp et_loop
```

Pentru alte cerințe

Pentru alte cerințe se schimbă doar corpul buclei. Numărul elementelor pare: `incl %ebx` în locul lui `addl`, cu `jnz et_next` după `testl`. Minimul: se inversează comparația și se pornește cu `v[0]`.

PARTEA V GREȘELI

10. Greșeli frecvente

Cum folosiți acest tabel

Unele comenzi sau „idei” par ok în esență, dar assembly are reguli stricte. Am extras aici niște greșeli foarte des întâlnite și eventual înainte să predați lucrarea sau dacă primiți erori la momentul rulării codului să vă verificați cu acest tabel înainte și să vă asigurați că țineți cont de tot ce scrie aici:

Instrucțiuni care par firești, dar sunt „ilegale”

Instrucțiune greșită	De ce nu merge	Varianta corectă
`movl x, y`	memorie → memorie. Cel mult **un** operand poate fi în memorie	`movl x, %eax` apoi `movl %eax, y`
`addl x, y`, `cmpl x, y`, `subl x, y`	aceeași regulă pentru toate instrucțiunile cu doi operanzi	trecere prin registru
`movl %eax, $5`	destinația nu poate fi constantă	`movl $5, %eax`
`cmpl %eax, $5`	la fel, `$5` nu poate fi al doilea operand	`cmpl $5, %eax`
`mull $5`, `divl $10`	`mul` și `div` nu acceptă constantă	`movl $5, %ebx` apoi `mull %ebx`
`movl %al, %ebx`	dimensiuni diferite (8 biți în 32)	`movzbl %al, %ebx` (completează cu 0) sau `movsbl` (cu semn)
`movb %eax, %bl`, `movl %ax, %ebx`	sufixul și registrele trebuie să aibă aceeași dimensiune	`movb %al, %bl`, `movzwl %ax, %ebx`
`movb %al, %sil`, `movb $1, %si`	`%esi` și `%edi` nu au parte de 8 biți pe 32 de biți	folosești `%al`...`%dl`
`movl v(,%edi,3), %eax`	factorul de scală poate fi doar 1, 2, 4 sau 8	`v(,%edi,4)`
`movl (%ax), %ebx`	adresele se formează doar cu registre de 32 de biți	`movl (%eax), %ebx`

Instrucțiune greșită	De ce nu merge	Varianta corectă
`incl $5`, `decl $1`	`inc` și `dec` lucrează pe un registru sau pe memorie	`incl %eax`
`shll %ebx, %eax`	numărul de poziții este constantă sau `%cl`	`shll %cl, %eax`
`lea %eax, %ebx`	sursa lui `lea` trebuie să fie o adresă de memorie	`lea (%eax), %ebx` sau `movl %eax, %ebx`
`pushb %al`	stiva lucrează cu 16 sau 32 de biți	`pushl %eax`
`mov $1, x` (fără sufix)	dimensiunea este ambiguă: assemblerul nu știe câți octeți scrie în `x`	`movl $1, x`

Lucruri care par ilegale, dar merg

- `movl $5, x` este corect (constantă → memorie), la fel și `addl $4, x`.
- `movl $x, %eax` este corect (adresa lui `x` într-un registru).
- `imull $5, %eax, %ebx` merge cu constanta, spre deosebire de `mull`.

Greșeli care apar cel mai des în lucrări

Greșelile care apar cel mai des în lucrări. Cele de logică nu sunt prinse de assembler: programul se compilează, dar dă rezultate greșite sau se blochează:

- Ordinea operanzilor inversată: în AT&T destinația este ultima.
- `cmpl a, b` comparat în ordine greșită: `jg` verifică b > a, nu a > b (ordinea operanzilor la `cmpl` interpretată invers).
- `$x` și `x` confundate: unul este adresa, celălalt valoarea din memorie.
- Lipsa ieșirii (`movl $1, %eax` / `int $0x80`).
- `divl` fără `xorl %edx, %edx` înainte.
- `incl %edi` uitat pe ramura care sare peste prelucrare, ceea ce produce buclă infinită.
- `loop` cu `%ecx` egal cu 0 la început, sau `%ecx` folosit și pentru altceva în buclă.
- Mutare memorie → memorie într-o singură instrucțiune (`movl x, y` nu există; se trece prin registru).